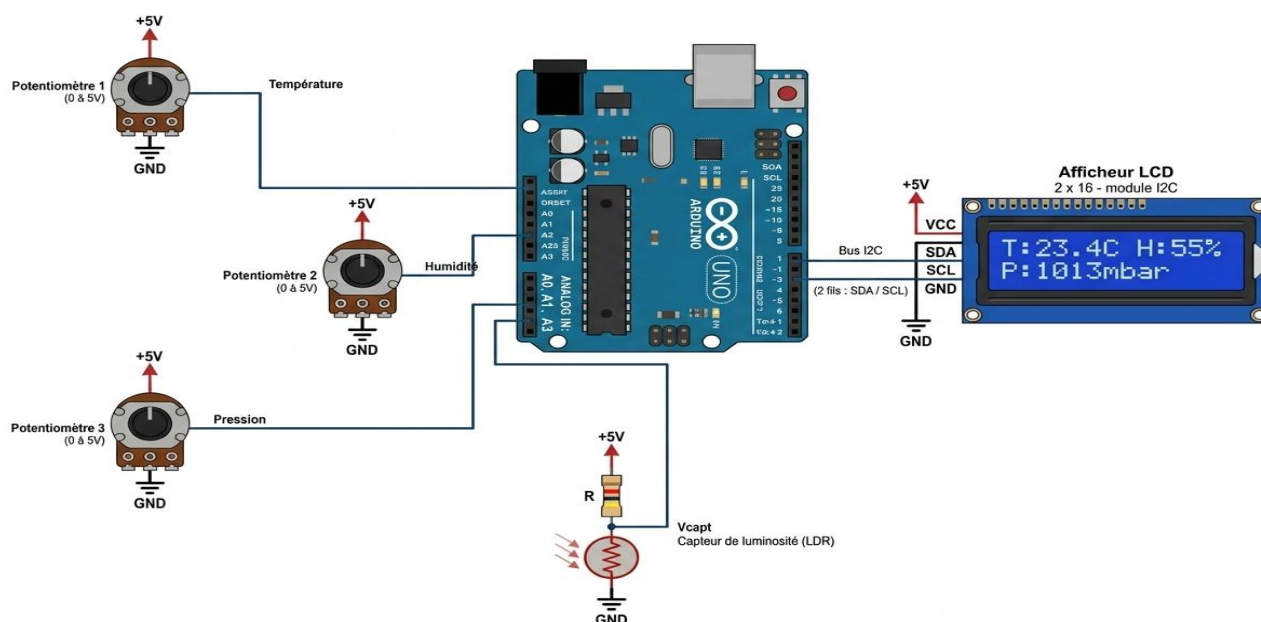


TP PROJET — STATION DE MESURES ENVIRONNEMENTALES

Terminale CIEL • Arduino UNO • entrées analogiques • LCD 16x2 I²C • ouverture Wi-Fi

Schéma de câblage - Station de mesure environnementale

Simulation des capteurs par potentiomètres + LDR - Arduino Uno - Afficheur LCD I2C



Légende :
Temp. : 0-5V → -10 à 40 degC | Humidité : 0-5V → 10% à 90% | Pression : 0-5V → 850 à 1100 mbar | Luminosité : seuil V_{capt} = 2,5V (jour si < 2,5V, nuit si ≥ 2,5V)

Projet : mesurer / simuler température, humidité, pression et luminosité, puis afficher les résultats.

Élément	Cahier des charges
Durée	4 séances de 4 h (16 h)
Carte	Arduino Uno (matériel réel dès la séance 1)
Capteurs simulés	3 potentiomètres 10 kΩ + 1 LDR + résistance 10 kΩ
Entrées	A0 température • A1 humidité • A2 pression • A3 luminosité
Affichage	LCD 16x2 avec interface I ² C
Méthode	Simulation Tinkercad AVANT tout câblage réel
Extension	Shield Wi-Fi → serveur Web / API / tableau de bord

Compétences visées

Analyser un schéma, câbler un système, mesurer une tension, exploiter un ADC, programmer en C/C++ Arduino, utiliser un bus I²C, tester et diagnostiquer un système, simuler avant de câbler (méthodologie de prototypage virtuel).

1. Présentation du projet

Le système représente une petite station de mesures. Pour rendre le projet accessible et reproductible en classe, trois capteurs sont simulés par des potentiomètres : température, humidité et pression. Une LDR permet de simuler la luminosité. L'Arduino mesure les tensions, les transforme en grandeurs physiques puis les affiche sur un LCD 16×2 I²C.

1.1 Pourquoi commencer par des potentiomètres ?

Un capteur réel fournit souvent une grandeur électrique exploitable par une électronique de mesure. Ici, le potentiomètre permet aux élèves de régler facilement une tension entre 0 et 5 V. On peut donc simuler différentes valeurs de capteurs sans acheter trois capteurs spécialisés.

1.2 Architecture fonctionnelle

Capteurs simulés → Acquisition analogique (ADC Arduino) → Conversion tension → grandeur physique → Traitement / seuils → LCD 16×2 I²C → Extension possible : Wi-Fi → serveur Web.

1.3 Répartition des séances

Séance	Contenu	Production attendue
1 — 4 h	Diviseur de tension (théorie complète) et capteur LDR : simulation Tinkercad puis câblage réel, bus I ² C et premier affichage LCD	État SOMBRE/CLAIR affiché sur le LCD 16×2 I ² C, en simulation ET sur carte réelle
2 — 4 h	Ajout du capteur température (A0) : droite d'étalonnage, extension du système existant	LDR + température affichées en alternance sur le LCD
3 — 4 h	Ajout des capteurs humidité (A1) et pression (A2) : conversions complètes	Les 4 grandeurs affichées en boucle sur le LCD et le port série
4 — 4 h	Intégration Wi-Fi : mise en réseau, serveur Web, tableau de bord	Système complet consultable à distance + compte rendu

2. Cahier des charges et modèles mathématiques

Grandeur	Entrée	0 V	5 V	Relation
Température	A0	-10 °C	40 °C	$T = 10 \cdot V - 10$
Humidité	A1	10 %	90 %	$H = 16 \cdot V + 10$
Pression	A2	850 mbar	1100 mbar	$P = 50 \cdot V + 850$
Luminosité	A3	—	—	$V < 2,5 \text{ V} \rightarrow \text{SOMBRE}$

Important

Ces relations sont des modèles pédagogiques imposés par le cahier des charges. Les potentiomètres ne mesurent pas réellement la température, l'humidité ou la pression. La LDR n'est pas étalonnée en lux.

2.1 De la valeur ADC à la tension

Avec une référence de 5 V et un ADC sur 10 bits :

```
N = analogRead(pin); // 0 à 1023
V = N * 5.0 / 1023.0; // tension en volts
```

N	V approximative
0	0 V
256	
	2,50 V
768	
1023	5 V

Résolution théorique sur 0–5 V : $5 / 1023 \approx 4,89$ mV par pas ADC.

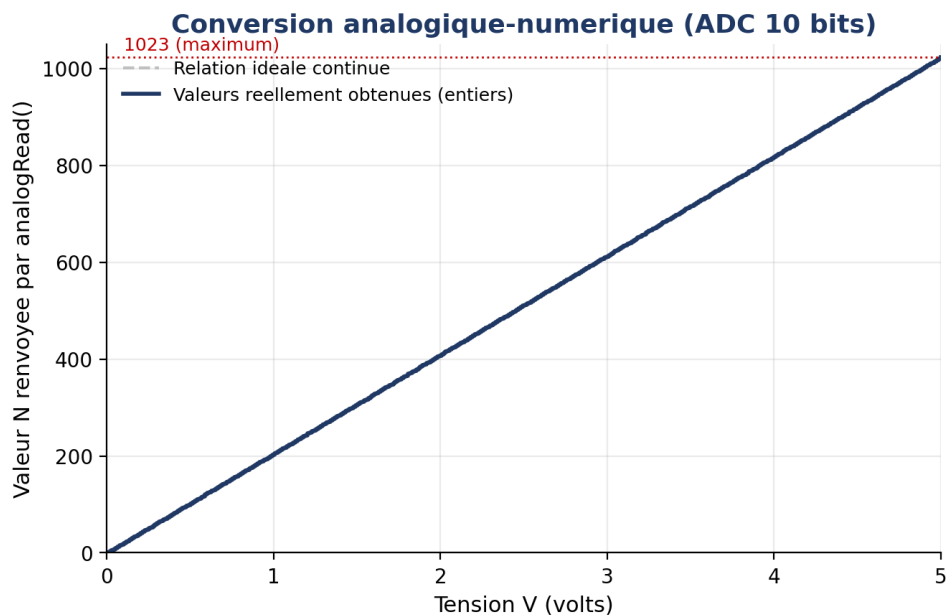


Figure — Relation entre la tension V et la valeur N renvoyée par analogRead().

2.2 D'où viennent les formules ? La droite d'étalonnage

Les relations $T = 10V - 10$, $H = 16V + 10$ et $P = 50V + 850$ ne sont pas magiques : ce sont des fonctions affines (droites) construites à partir de DEUX points de mesure connus, exactement comme on étalonnerait un vrai capteur en laboratoire.

Méthode générale : on connaît la grandeur physique attendue pour deux tensions particulières, $V = 0$ V et $V = 5$ V (les deux colonnes du tableau du §2). On cherche une relation du type $Y = a \cdot V + b$, avec :

```
a = (Ymax - Ymin) / (Vmax - Vmin)
b = Ymin - a x Vmin
```

Exemple détaillé : la température

Deux points connus : à $V = 0$ V, $T = -10$ °C ; à $V = 5$ V, $T = 40$ °C.

$$a = (40 - (-10)) / (5 - 0) = 50 / 5 = 10$$

$$b = -10 - (10 \times 0) = -10$$

$$\text{donc : } T = 10 \times V - 10$$

C'est exactement la méthode d'étalonnage utilisée en instrumentation réelle

Sur un vrai capteur, on relève sa sortie pour deux valeurs connues de la grandeur physique (par exemple à 0 °C dans de la glace fondante, puis à 100 °C dans de l'eau bouillante), puis on calcule a et b de la même façon. C'est ce qu'on appelle une droite d'étalonnage ou droite de calibration.

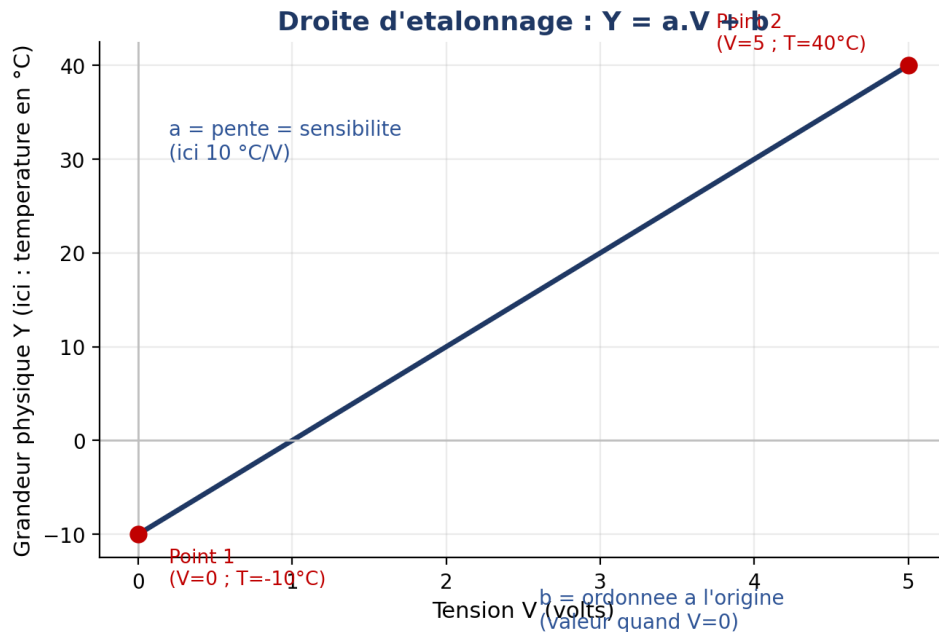


Figure — La droite d'étalonnage : deux points connus suffisent à définir a (pente) et b (ordonnée à l'origine).

À votre tour : retrouver les formules de l'humidité et de la pression

En utilisant la même méthode et les valeurs du tableau du §2, calculer a et b pour l'humidité puis pour la pression, et vérifier que vous retombez bien sur les formules données :

Grandeur	Point 1 (V=0)	Point 2 (V=5)	$a = (Y_{\max} - Y_{\min}) / 5$	$b = Y_{\min} - a \times 0$	Formule $Y = a \cdot V + b$

2.3 Et si c'étaient de vrais capteurs ?

Les potentiomètres et la LDR permettent de s'entraîner sans matériel coûteux, mais dans un vrai système embarqué, ce sont de véritables capteurs qui produiraient une tension (ou une résistance) liée à la grandeur physique. Le principe reste le même : une caractéristique électrique, une droite d'étalonnage, puis une conversion dans le programme.

Grandeur	Exemple de capteur réel	Grandeur électrique produite	Relation typique
Température	LM35 (capteur analogique)	Tension proportionnelle à T	$V_{out} = 10 \text{ mV} / ^\circ\text{C}$ (relation déjà linéaire, sans diviseur)
Température	Thermistance CTN + diviseur de tension	Résistance qui varie avec T	Relation non linéaire (équation de Steinhart-Hart), plus complexe qu'une droite
Humidité	Capteur capacitif d'humidité (type HIH-4000)	Tension proportionnelle à l'humidité relative	V_{out} quasi linéaire entre 0 et 100 % HR
Pression	Capteur piézorésistif (type MPX4115)	Tension proportionnelle à la pression atmosphérique	$V_{out} = V_s \times (0,009 \times P - 0,095)$, une droite comme celle du §2.2
Luminosité	LDR (déjà utilisée dans ce projet)	Résistance qui diminue avec la lumière	Non linéaire, utilisée ici surtout comme un seuil

Ce qu'il faut retenir

Un capteur « de base » (LM35, capteur de pression piézorésistif...) transforme une grandeur physique en une tension, souvent selon une relation affine $Y = a \cdot V + b$, tout comme nos potentiomètres. La différence est que a et b sont fixés par la physique du capteur (donnés dans sa documentation technique / datasheet), et non choisis arbitrairement par le cahier des charges.

3. Rappels d'électricité indispensables

3.1 Tension, courant et résistance

Grandeur	Symbole	Unité	Définition pédagogique
Tension	U	V	Différence de potentiel entre deux points
Courant	I	A	Déplacement de charges électriques dans un circuit
Résistance	R	Ω	Propriété qui s'oppose au passage du courant

Loi d'Ohm : $U = R \times I$. On peut aussi écrire $I = U/R$ et $R = U/I$.

3.2 Pourquoi mesurer une tension ?

Une entrée analogique Arduino mesure une tension par rapport à la masse GND. Le potentiomètre transforme donc une position mécanique en tension électrique. Le programme récupère ensuite cette tension sous forme numérique.

3.3 Potentiomètre

Le potentiomètre possède trois bornes. Les deux extrémités sont reliées à +5 V et GND. Le curseur fournit une tension variable. Dans ce projet, trois potentiomètres jouent le rôle de sorties de capteurs. Comme on le voit au paragraphe suivant, un potentiomètre est en réalité un diviseur de tension réglable.

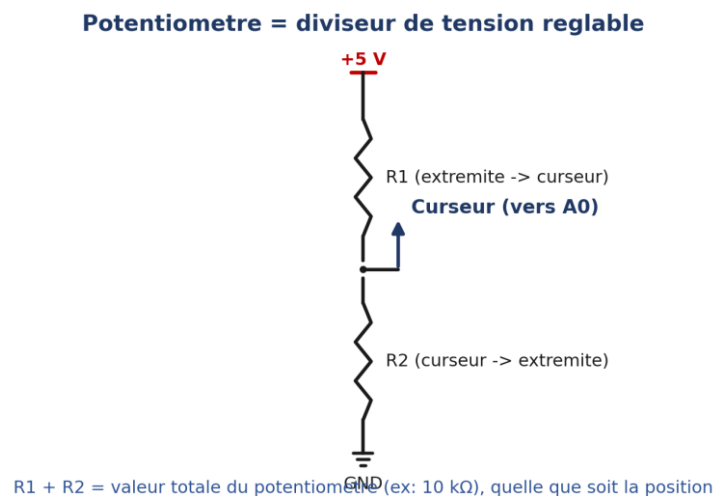


Figure — Le potentiomètre vu comme un diviseur de tension réglable : $R1+R2$ reste constant, seul le rapport change.

3.4 Le diviseur de tension

Un diviseur de tension est le montage électronique de base le plus utilisé pour transformer une résistance variable en tension mesurable par l'Arduino. Il est constitué de deux résistances $R1$ et $R2$ placées en série entre +5 V et GND. La tension de sortie V_{out} est prélevée au point milieu, entre les deux résistances.

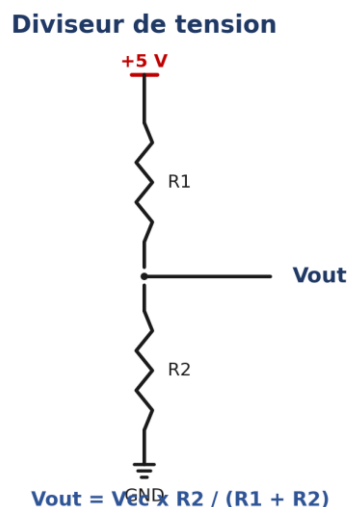


Figure — Schéma électrique du diviseur de tension générique.

Formule du diviseur de tension : $V_{out} = V_{cc} \times R2 / (R1 + R2)$

Exemple de calcul

Avec $V_{cc} = 5\text{ V}$, $R_1 = 10\text{ k}\Omega$ et $R_2 = 10\text{ k}\Omega$:

$$V_{out} = 5 \times 10 / (10 + 10) = 5 \times 0,5 = 2,5\text{ V}$$

Le potentiomètre EST un diviseur de tension

Dans un potentiomètre, la piste résistive totale ($10\text{ k}\Omega$) est coupée en deux par le curseur : R_1 = portion entre $+5\text{ V}$ et le curseur, R_2 = portion entre le curseur et GND, avec toujours $R_1 + R_2 = 10\text{ k}\Omega$.

Tourner le bouton ne change pas la résistance totale : cela change seulement le partage entre R_1 et R_2 , donc la valeur de V_{out} . C'est exactement le calcul qui sera vérifié à la main puis au multimètre en séance 1 (§5.1).

3.5 LDR et diviseur de tension : la position change TOUT

La résistance d'une LDR diminue quand la lumière augmente (typiquement quelques centaines d'ohms en pleine lumière, plusieurs dizaines de $\text{k}\Omega$ dans le noir). Associée à une résistance fixe de $10\text{ k}\Omega$, elle forme un autre exemple de diviseur de tension (vu en 3.4). Mais attention : selon que l'on place la LDR en haut (côté $+5\text{ V}$) ou en bas (côté GND) du diviseur, le sens de variation de V_{out} s'inverse complètement !

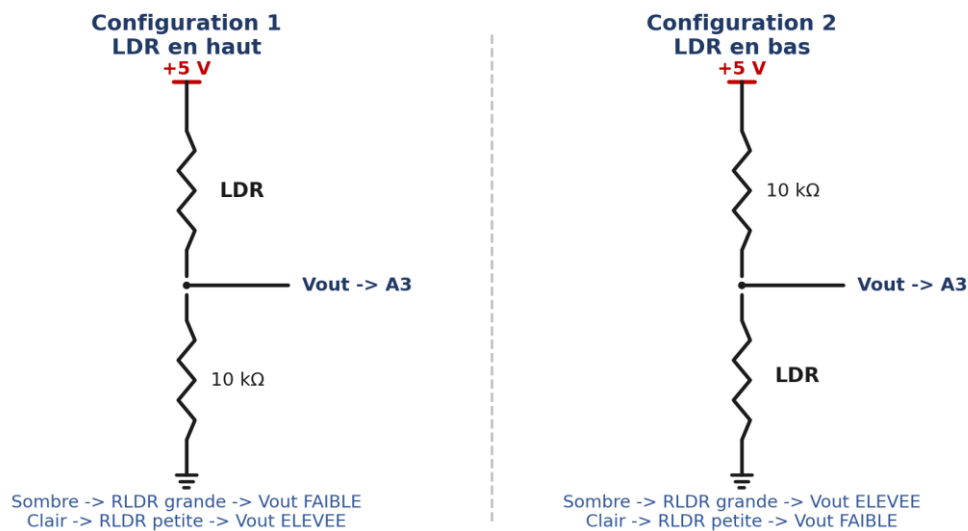
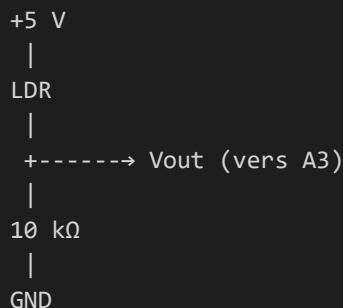


Figure — Les deux configurations possibles de la LDR et leur effet opposé sur V_{out} .

Configuration 1 — LDR en haut

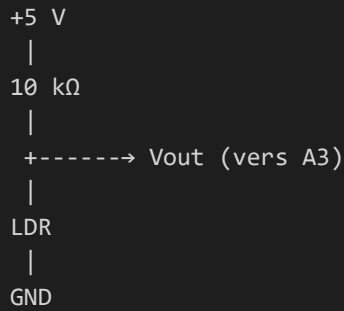


Ici $V_{out} = V_{cc} \times R_{fixe} / (R_{LDR} + R_{fixe})$. Avec $R_{fixe} = 10\text{ k}\Omega$:

Ambiance	RLDR (estimation)	Calcul	Vout
Pleine lumière	$\approx 1\text{ k}\Omega$	$5 \times 10 / (1 + 10)$	$\approx 4,5\text{ V}$ (haute)
Obscurité	$\approx 50\text{ k}\Omega$	$5 \times 10 / (50 + 10)$	$\approx 0,8\text{ V}$ (basse)

Dans cette configuration : plus il fait sombre, plus Vout est FAIBLE. Plus il fait clair, plus Vout est ÉLEVÉE.

Configuration 2 — LDR en bas



Ici $V_{out} = V_{cc} \times R_{LDR} / (R_{fixe} + R_{LDR})$. Avec $R_{fixe} = 10 \text{ k}\Omega$:

Ambiance	RLDR (estimation)	Calcul	Vout
Pleine lumière	$\approx 1 \text{ k}\Omega$	$5 \times 1 / (10 + 1)$	$\approx 0,5 \text{ V}$ (basse)
Obscurité	$\approx 50 \text{ k}\Omega$	$5 \times 50 / (10 + 50)$	$\approx 4,2 \text{ V}$ (haute)

Dans cette configuration, c'est l'inverse : plus il fait sombre, plus Vout est ÉLEVÉE. Plus il fait clair, plus Vout est FAIBLE.

Une seule configuration est compatible avec le cahier des charges

Le cahier des charges impose la règle : $V < 2,5 \text{ V} \rightarrow \text{SOMBRE}$ (voir tableau du §2). Or dans la configuration 2 (LDR en bas), l'obscurité donne une tension HAUTE ($\approx 4,2 \text{ V}$), ce qui contredirait cette règle !

Seule la configuration 1 (LDR en haut, résistance fixe en bas) donne une tension qui BAISSÉ quand il fait sombre, donc qui respecte bien la règle $V < 2,5 \text{ V} \rightarrow \text{SOMBRE}$. La position du composant n'est donc pas un détail : c'est elle qui donne du sens à la mesure.

Question d'étude

- Sans regarder la réponse ci-dessus, refaites le calcul de Vout pour les deux configurations avec $R_{LDR} = 5 \text{ k}\Omega$ (luminosité intermédiaire) et vérifiez que le sens de variation annoncé est cohérent.

4. Étude des composants

Composant	Fonction	Points à connaître
Arduino Uno	Acquisition et traitement	Microcontrôleur, ADC 10 bits, E/S numériques
Potentiomètre $10 \text{ k}\Omega \times 3$	Simulation de 3 capteurs	Résistance variable, curseur, 0–5 V
LDR	Simulation de luminosité	Résistance dépendante de la lumière
Résistance $10 \text{ k}\Omega$	Avec la LDR	Diviseur de tension
LCD $16 \times 2 \text{ I}^2\text{C}$	Affichage	16 caractères $\times$ 2 lignes, bus I^2C
Breadboard	Montage sans soudure	Rails d'alimentation, connexions
Fils Dupont	Liaisons	Respecter +5 V / GND / signaux

Questions d'étude

- Quel est le rôle des deux bornes extrêmes d'un potentiomètre ?
- Pourquoi le curseur peut-il fournir une tension comprise entre 0 et 5 V ?
- Dans la formule $V_{out} = V_{cc} \times R_2 / (R_1 + R_2)$, que devient V_{out} si $R_2 = 0 \Omega$? Et si $R_1 = 0 \Omega$?
- Pourquoi ne faut-il jamais appliquer une tension supérieure à 5 V à une entrée analogique dans ce montage ?
- Quel est le rôle de la résistance de 10 k Ω avec la LDR ?
- Entre les deux configurations du §3.5, laquelle utiliserez-vous pour respecter le cahier des charges, et pourquoi ?
- Pourquoi toutes les masses doivent-elles être communes ?
- Quelle différence entre un capteur réel et un potentiomètre utilisé comme simulateur ?

5. SÉANCE 1 — Diviseur de tension, LDR et premier affichage LCD

Durée : 4 h | Objectif de cette séance

Comprendre en profondeur le diviseur de tension (le calculer ET le vérifier au multimètre), comprendre pourquoi la position d'une résistance variable dans le diviseur change tout, et mettre en œuvre le premier capteur du projet : la LDR (luminosité), affichée sur un écran LCD I²C.

Chaque binôme reçoit aujourd'hui : une carte Arduino Uno, une breadboard, une LDR, une résistance 10 kΩ, un écran LCD 16×2 avec module I²C, des fils Dupont et un câble USB. Aucun potentiomètre n'est utilisé aujourd'hui : les potentiomètres (température, humidité, pression) arriveront aux séances suivantes.

Méthode : on calcule et on vérifie AVANT de simuler, on simule sur Tinkercad AVANT de câbler en réel.

5.0 Déroulé horaire détaillé (4 h = 240 min)

Horaire	Durée	Activité	Livrable
0h00 – 0h15	15 min	Accueil, objectifs, distribution du matériel, consignes de sécurité	—
0h15 – 0h35	20 min	Rappels U/I/R	Réponses au quiz
0h35 – 1h05	30 min	Le diviseur de tension : théorie, formule, mesure de RLDR au multimètre	RLDR mesurée en lumière/obscurité
1h05 – 1h35	30 min	Les deux configurations de la LDR : calcul prédictif et choix cohérent avec le cahier des charges	Configuration validée
1h35 – 1h50	15 min	Découverte du bus I ² C et du module LCD I ² C	Schéma annoté
1h50 – 2h20	30 min	PARTIE A — Tinkercad : montage LDR + LCD, lecture et conversion	Circuit + code de lecture validés
2h20 – 2h35	15 min	Pause / vérification du travail par le professeur (visa simulation)	Simulation validée
2h35 – 2h50	15 min	PARTIE A (suite) — Seuil SOMBRE/CLAIR et affichage LCD en simulation	Programme simulé validé
2h50 – 3h10	20 min	PARTIE B1 — Câblage réel : LDR + LCD I ² C	Montage réel câblé
3h10 – 3h30	20 min	PARTIE B2 — Scan I ² C, bibliothèque, vérification au multimètre	LCD réel allumé et testé
3h30 – 3h50	20 min	PARTIE B3 — Programme final réel, comparaison simulation / réel	Système LDR+LCD fonctionnel
3h50 – 4h00	10 min	Fiche de compte rendu et synthèse	Fiche remplie

Règle d'or de la séance

Aucun câblage réel ni branchement USB tant que le circuit Tinkercad n'a pas été simulé avec succès et montré au professeur.

5.1 Rappels rapides (20 min)

1. Que mesure une entrée analogique Arduino : une tension, un courant ou une résistance ?
2. Sur combien de bits l'ADC de l'Arduino Uno code-t-il une tension ? Quel est l'intervalle de valeurs obtenu ?

3. Pourquoi ne faut-il jamais relier directement +5 V à GND ?

Le professeur valide oralement avant de continuer.

5.2 Le diviseur de tension : calculer PUIS vérifier (30 min)

Comme vu en 3.4, un diviseur de tension est formé de deux résistances R1 (côté +5 V) et R2 (côté GND) en série. La tension prélevée entre les deux, Vout, suit la formule :

$$V_{out} = V_{cc} \times R2 / (R1 + R2)$$

Aujourd'hui, R2 sera la LDR elle-même : sa résistance change selon la lumière ambiante. On va d'abord la mesurer directement, sans aucun câblage électrique, juste avec le multimètre en mode ohmmètre.

Manip — mesurer RLDR au multimètre

Poser les deux pointes du multimètre (mode Ω) directement sur les deux pattes de la LDR. Aucune alimentation nécessaire : la LDR est un composant passif.

4. Mesurer RLDR en pleine lumière (sous l'éclairage normal de la salle).
5. Recouvrir complètement la LDR avec la main ou un tissu opaque et mesurer à nouveau RLDR.
6. Mesurer aussi la valeur de la résistance fixe 10 k Ω distribuée (elle doit être proche de 10 000 Ω).

Condition	RLDR mesurée (Ω ou k Ω)
Ce que l'on observe RLDR est petite en pleine lumière (souvent quelques centaines d'ohms à quelques k Ω) et grande dans le noir (souvent plusieurs dizaines de k Ω). C'est l'inverse d'une résistance fixe : elle varie toute seule selon la lumière.	

5.3 Les deux configurations de la LDR : la position change TOUT (30 min)

Comme détaillé en 3.5, selon que l'on place la LDR en haut (côté +5 V) ou en bas (côté GND) du diviseur, le sens de variation de Vout s'inverse. Reprenons le calcul avec vos propres valeurs mesurées de RLDR.

Configuration 1 — LDR en haut, résistance fixe en bas

```
+5 V
|
LDR
|
+-----> Vout (vers A3)
|
10 k $\Omega$ 
|
GND
```

$V_{out} = V_{cc} \times R_{fixe} / (RLDR + R_{fixe})$. Calculer Vout pour vos deux mesures de RLDR (lumière et obscurité) :

Condition	RLDR (Ω)	Vout calculée (V) — Configuration 1

Configuration 2 — LDR en bas, résistance fixe en haut

```

+5 V
|
10 k $\Omega$ 
|
+-----> Vout (vers A3)
|
LDR
|
GND

```

$V_{out} = V_{cc} \times RLDR / (R_{fixe} + RLDR)$. Calculer V_{out} pour les mêmes conditions :

Condition	RLDR (Ω)	Vout calculée (V) — Configuration 2

Une seule configuration est compatible avec le cahier des charges

Le cahier des charges impose la règle : $V < 2,5 \text{ V} \rightarrow \text{SOMBRE}$ (voir tableau du §2). D'après vos calculs, quelle configuration donne une tension qui BAISSÉ quand il fait sombre ? C'est celle-là qu'il faudra câbler.

- D'après vos mesures, quelle configuration retenez-vous : Configuration 1 ou Configuration 2 ?
- Justifier en une phrase, en utilisant vos propres valeurs calculées ci-dessus.

5.4 Découverte du bus I²C et du module LCD I²C (15 min)

Un écran LCD 16×2 « classique » nécessite normalement 6 à 10 fils pour fonctionner. Le bus I²C permet de réduire cela à seulement 2 fils de communication (en plus de l'alimentation) :

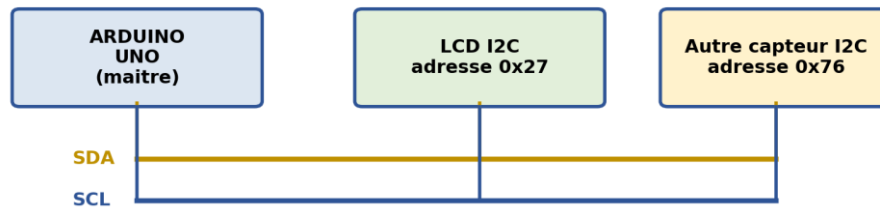
- SDA (Serial DATA) : la ligne sur laquelle circulent les données
- SCL (Serial CLock) : la ligne d'horloge qui cadence les échanges

Sur l'Arduino Uno, ces deux lignes sont fixes : SDA = broche A4, SCL = broche A5. Retournez l'écran LCD fourni : un petit circuit imprimé (module I²C, basé sur la puce PCF8574) est soudé au dos.

Élément du module	Rôle
4 broches VCC / GND / SDA / SCL	Alimentation et bus de communication

Élément du module	Rôle
Petit potentiomètre bleu (trimmer)	Réglage manuel du contraste de l'affichage
Adresse I ² C (0x27 ou 0x3F le plus souvent)	Identifiant unique du module sur le bus, à découvrir avec un scanner

Le bus I²C : plusieurs périphériques, une adresse chacun



*Un seul bus à 2 fils (SDA/SCL) relie plusieurs périphériques.
Chaque périphérique répond seulement quand son adresse est appelée.*

Figure — Le bus I²C : deux fils partagés (SDA/SCL), chaque périphérique répond à sa propre adresse.

5.5 PARTIE A — Simulation sur Tinkercad (1h15)

5.5.1 Montage du circuit virtuel (30 min)

Ajouter dans un nouveau circuit Tinkercad : 1 Arduino Uno, 1 LDR, 1 résistance 10 kΩ, 1 LCD 16×2 (avec option I²C activée). Câbler selon la Configuration 1 (validée au §5.3) :

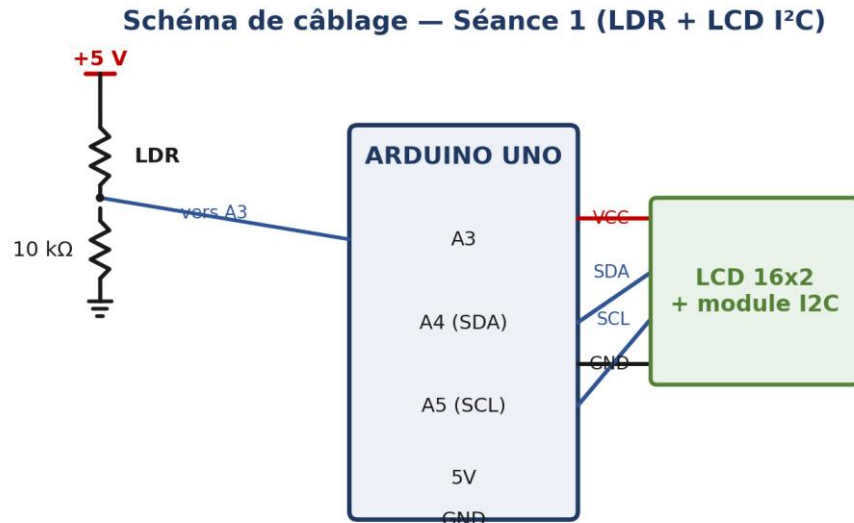


Figure — Schéma de câblage cible de la séance 1 : LDR (diviseur) + LCD I²C sur l'Arduino Uno.

Branchement	Broche Arduino
+5 V → LDR → point milieu → 10 kΩ → GND	Point milieu → A3
LCD I ² C : VCC / GND / SDA / SCL	5V / GND / A4 / A5

Branchement	Broche Arduino
<pre> const int PIN_LUM = A3; void setup() { Serial.begin(9600); } void loop() { int N = analogRead(PIN_LUM); float V = N * 5.0 / 1023.0; Serial.print("N="); Serial.print(N); Serial.print(" V="); Serial.println(V, 2); delay(500); } </pre>	

Simuler, ouvrir le moniteur série, cliquer sur la LDR virtuelle pour faire varier la luminosité et vérifier que N et V changent dans le bon sens (Configuration 1 : sombre → V faible).

5.5.2 Seuil SOMBRE/CLAIR et affichage LCD (15 min)

```

#include <Wire.h>
#include
<LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27,
16, 2);
const int PIN_LUM = A3;

void setup() {
  lcd.init();
  lcd.backlight();
  Serial.begin(9600);
}

void loop() {
  int N =
analogRead(PIN_LUM);
  float V = N * 5.0 /
1023.0;
  String etat = (V < 2.5) ?
"SOMBRE" : "CLAIR ";

  lcd.setCursor(0, 0);
  lcd.print("Luminosite:");
  lcd.setCursor(0, 1);
  lcd.print(etat);

  Serial.print("N=");
  Serial.print(N);
  Serial.print("  V=");
  Serial.print(V, 2);

```

```
Serial.print(" -> ");
Serial.println(etat);
delay(500);
}
```

Instruction	Explication
String etat = ...;	Crée une variable de type texte (chaîne de caractères)
(V < 2.5) ? "SOMBRE" : "CLAIR "	Opérateur ternaire : si la condition est vraie, renvoie la 1 ^{re} valeur, sinon la 2 ^e

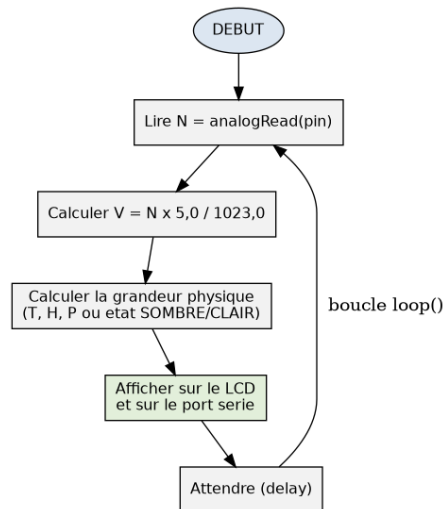


Figure — Algorithme de la boucle loop() : lire, convertir, afficher.

Simuler : faire varier la LDR virtuelle et vérifier que le LCD affiche bien SOMBRE ou CLAIR de façon cohérente avec la Configuration 1.

Validation professeur avant de continuer

Montrer au professeur le circuit Tinkercad complet (LDR + LCD) et l'affichage SOMBRE/CLAIR cohérent avec vos calculs du §5.3.

5.6 PARTIE B — Câblage et tests réels (1h00)

5.6.1 Matériel distribué à chaque binôme

Quantité	Matériel
1	Carte Arduino Uno
1	Câble USB A / B
1	Breadboard
1	LDR
1	Résistance 10 kΩ
1	Écran LCD 16×2 avec module I ² C soudé
7	Fils Dupont mâle-mâle
1	Multimètre (partagé par îlot)

5.6.2 Câblage réel — Configuration 1 (20 min)

- +5 V → LDR → point milieu → A3
- Point milieu → résistance 10 kΩ → GND
- LCD I²C : VCC → +5 V, GND → GND, SDA → A4, SCL → A5

Sécurité avant de brancher l'USB

Vérifier deux fois le câblage avant de connecter le câble USB : un +5V relié directement à GND peut endommager la carte.

5.6.3 Scanner I²C, bibliothèque et vérification au multimètre (20 min)

7. Installer la bibliothèque « LiquidCrystal I2C » (Frank de Brabander) via Outils > Gérer les bibliothèques.
8. Téléverser le programme scanner I²C ci-dessous pour trouver l'adresse réelle du module :

```
#include <Wire.h>

void setup() {
  Wire.begin();
  Serial.begin(9600);
  Serial.println("Scan I2C en cours...");
  for (byte adresse = 1; adresse < 127; adresse++) {
    Wire.beginTransmission(adresse);
    byte erreur = Wire.endTransmission();
    if (erreur == 0) {
      Serial.print("Peripherique trouve a l'adresse 0x");
      Serial.println(adresse, HEX);
    }
  }
}

void loop() {
}
```

Adresse I²C trouvée (ex : 0x27)

Mesurer maintenant la tension Vout réelle (entre A3 et GND) en pleine lumière et dans l'obscurité, et comparer à vos prédictions du §5.3 :

Condition	Vout mesurée au multimètre (V)	Vout prédite au §5.3 (V)

5.6.4 Programme final réel (20 min)

9. Recopier le programme final du §5.5.2 en adaptant l'adresse I²C si besoin.

10. Téléverser et ouvrir le moniteur série réel.

11. Couvrir puis découvrir la LDR réelle et vérifier que le LCD affiche SOMBRE puis CLAIR.

Condition	N réel	V réel	État affiché sur LCD réel

5.7 Travail élève — Fiche de compte rendu

Questions de synthèse

12. Écrire la formule du diviseur de tension et expliquer pourquoi la position de la LDR (en haut ou en bas) change le sens de variation de V_{out} .
13. Quelle configuration avez-vous retenue ? Pourquoi est-ce la seule cohérente avec la règle $V < 2,5\text{ V} \rightarrow$ SOMBRE ?
14. Comparer V_{out} prédite et V_{out} mesurée au multimètre (§5.6.3) : sont-elles proches ? Que peut expliquer un éventuel écart ?
15. Quel est l'intérêt du bus I²C par rapport à un câblage parallèle classique pour un écran LCD ?
16. Que représente concrètement la variable N ? Et la variable V ? Et la variable état ?

Checklist de fin de séance

- ☐ RLDR mesurée en lumière et en obscurité, résistance fixe vérifiée
- ☐ V_{out} calculée pour les 2 configurations, configuration cohérente choisie et justifiée
- ☐ Circuit Tinkercad (LDR + LCD) monté et simulé avec succès
- ☐ Circuit validé par le professeur
- ☐ Câblage réel réalisé et vérifié au multimètre
- ☐ Bibliothèque LiquidCrystal_I2C installée, adresse I²C trouvée
- ☐ Programme final téléversé, SOMBRE/CLAIR affiché correctement sur le LCD réel
- ☐ Questions de synthèse répondues

5.8 Pour aller plus loin (différenciation)

- Afficher aussi la tension V (avec 2 décimales) sur la 2^e ligne, à côté de l'état SOMBRE/CLAIR.
- Ajouter un seuil intermédiaire : SOMBRE / PENOMBRE / CLAIR avec deux seuils de tension.
- Essayer de câbler la Configuration 2 et vérifier expérimentalement que le comportement s'inverse bien comme prévu.

5.9 Dépannage — Que faire si...

Symptôme	Cause probable	Solution
L'état affiché ne change jamais	LDR mal câblée ou mauvaise configuration	Revérifier le câblage LDR + 10 kΩ selon la Configuration 1
Le LCD reste totalement noir/éteint	Pas d'alimentation ou câblage VCC/GND inversé	Vérifier VCC → 5V et GND → GND
Rétroéclairage allumé mais aucun texte	Contraste mal réglé	Tourner le petit potentiomètre bleu au dos du module
Le scanner I ² C ne trouve aucune adresse	SDA/SCL inversés ou mal branchés	Vérifier SDA → A4 et SCL → A5
N reste bloqué à 0 ou 1023	Point milieu non relié à A3	Vérifier le câblage du point milieu du diviseur
Carte non reconnue	Câble USB de mauvaise qualité ou pilote manquant	Changer de câble, vérifier Outils > Type de carte

5.10 Grille d'auto-évaluation / compétences

Compétence	Non acquis	En cours	Acquis	Expert
Expliquer et calculer un diviseur de tension				
Mesurer une résistance variable au multimètre				
Choisir et justifier une configuration de câblage				
Câbler un module LCD I ² C (SDA/SCL)				
Utiliser une variable texte et l'opérateur ternaire				
Comparer simulation et mesure réelle, expliquer un écart				

Prochaine séance : ajout du capteur de température (potentiomètre sur A0), méthode de la droite d'étalonnage.

6. SÉANCE 2 — Le capteur de température

Durée : 4 h | Ce qui change par rapport à la séance 1

Le montage LDR + LCD de la séance 1 reste en place. On ajoute aujourd'hui un potentiomètre sur A0, qui simule un capteur de température, et on apprend la méthode générale pour retrouver une formule de conversion à partir de deux points connus (droite d'étalonnage).

Même méthode qu'en séance 1 : on calcule et prédit AVANT de simuler, on simule sur Tinkercad AVANT de câbler en réel.

6.0 Déroulé horaire détaillé (4 h = 240 min)

Horaire	Durée	Activité	Livrable
0h00 – 0h10	10 min	Accueil, rappel séance 1 (diviseur, LDR, LCD), objectifs	—
0h10 – 0h30	20 min	La droite d'étalonnage : méthode générale + exemple température (§2.2)	Calcul de a et b refait à la main
0h30 – 1h00	30 min	PARTIE A1 — Tinkercad : ajout du potentiomètre température au circuit existant	Circuit Tinkercad mis à jour
1h00 – 1h40	40 min	PARTIE A2 — Codage : lecture, conversion, affichage combiné Temp + Luminosité	Programme simulé validé
1h40 – 2h00	20 min	Trace à la main et interprétation des coefficients	Tableau de trace complété
2h00 – 2h15	15 min	Pause / vérification du travail par le professeur (visa simulation)	Simulation validée
2h15 – 2h45	30 min	PARTIE B1 — Câblage réel du potentiomètre température	Montage réel câblé
2h45 – 3h05	20 min	PARTIE B2 — Vérification au multimètre + téléversement	Programme téléversé
3h05 – 3h35	30 min	PARTIE B3 — Tests, comparaison simulation / réel	Système Temp + LDR fonctionnel
3h35 – 4h00	25 min	Fiche de compte rendu et synthèse	Fiche remplie

Toujours la même règle d'or

Aucun câblage réel supplémentaire ni nouveau branchement USB tant que le montage complet (LDR + température + LCD) n'a pas été simulé avec succès sur Tinkercad et validé par le professeur.

6.1 La droite d'étalonnage : méthode et application à la température (20 min)

Comme vu en 2.2, la relation entre la tension V et une grandeur physique Y est une droite $Y = a \cdot V + b$. On connaît deux points ($V=0$ et $V=5$ V, voir cahier des charges §2) et on calcule :

$$a = (Y_{\max} - Y_{\min}) / (V_{\max} - V_{\min})$$
$$b = Y_{\min} - a \times V_{\min}$$

Refaire ce calcul, sans regarder le §2.2, pour la température ($0 \text{ V} \rightarrow -10 \text{ }^{\circ}\text{C}$; $5 \text{ V} \rightarrow 40 \text{ }^{\circ}\text{C}$) :

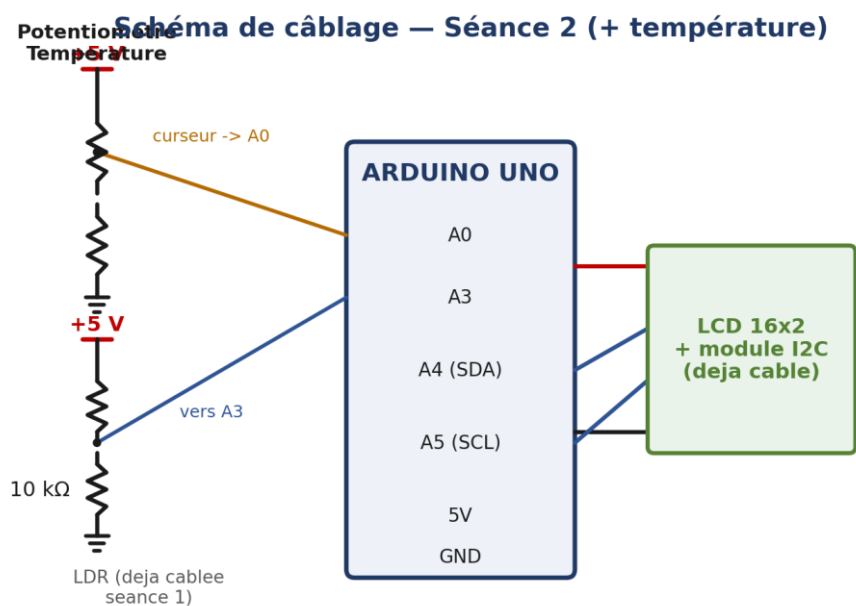
a =	b =	Formule finale T = a·V + b
Interprétation physique a est la sensibilité du capteur simulé : ici, 10 °C par volt. b est la valeur affichée quand V = 0 V : ici, -10 °C. C'est exactement la méthode utilisée pour étalonner un vrai capteur en laboratoire.		

6.2 PARTIE A — Simulation sur Tinkercad (1h40)

6.2.1 Ajout du potentiomètre au circuit existant (30 min)

Reprendre le circuit Tinkercad de la séance 1 (LDR + LCD déjà câblés) et ajouter 1 « Rotation Potentiometer » :

Composant	Branchement	Broche Arduino
Potentiomètre température	Extrémité 1	5V
Potentiomètre température	Extrémité 2	GND
Potentiomètre température	Curseur	A0
LDR + LCD (déjà câblés séance 1)	—	A3 / A4 / A5



En orange : câblage nouveau de la séance 2. Le reste (LDR, LCD) était déjà en place.

Figure — Ajout du potentiomètre température (en orange) sur le montage LDR + LCD déjà câblé.

6.2.2 Codage — lecture, conversion et affichage combiné (40 min)

Le LCD a 2 lignes : parfait pour afficher 2 capteurs en même temps, sans avoir besoin d'alternier. Ligne 1 = température, ligne 2 = luminosité.

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);
```

```

const int PIN_TEMP = A0;
const int PIN_LUM = A3;

void setup() {
  lcd.init();
  lcd.backlight();
  Serial.begin(9600);
}

void loop() {
  int Nt = analogRead(PIN_TEMP);
  int Nl = analogRead(PIN_LUM);

  float Vt = Nt * 5.0 / 1023.0;
  float Vl = Nl * 5.0 / 1023.0;

  float T = 10.0 * Vt - 10.0;          // droite d'etalonnage
  String etat = (Vl < 2.5) ? "SOMBRE" : "CLAIR ";

  lcd.setCursor(0, 0);
  lcd.print("Temp: "); lcd.print(T, 1); lcd.print(" C  ");
  lcd.setCursor(0, 1);
  lcd.print("Lum: "); lcd.print(etat);

  Serial.print("T="); Serial.print(T, 1);
  Serial.print(" Etat="); Serial.println(etat);
  delay(500);
}

```

Tester en simulation : faire varier le potentiomètre température virtuel et vérifier que la ligne 1 du LCD change, pendant que la ligne 2 (luminosité) reste indépendante.

Position potentiomètre	N simulation	V calculée	T calculée (°C)

6.3 Trace à la main et interprétation (20 min)

Sans lancer la simulation, calculer à la main V et T pour N = 614, puis vérifier avec Tinkercad :

N (donné)	V = $N \times 5 / 1023$ (calcul)	T = $10V - 10$ (calcul)	Résultat vérifié en simulation

- Que représente concrètement la variable Nt ? Et Vt ? Et T ?
- Si on augmentait la sensibilité a de 10 à 20 °C/V (en gardant b = -10), la température varierait-elle plus vite ou plus lentement quand on tourne le potentiomètre ?

✓ Validation professeur avant de continuer

Montrer au professeur le circuit Tinkercad complet (LDR + température + LCD) avec les 2 lignes cohérentes.

6.4 PARTIE B — Câblage et tests réels (1h20)

6.4.1 Matériel supplémentaire distribué

Quantité	Matériel
1	Potentiomètre 10 k Ω (température)
3	Fils Dupont mâle-mâle supplémentaires

6.4.2 Câblage réel (30 min)

- Extrémité 1 → +5 V, Extrémité 2 → GND, Curseur → A0

⚠ Sécurité

Débrancher l'USB avant toute modification du câblage.

6.4.3 Vérification au multimètre et téléversement (20 min)

Mesurer la tension du curseur pour 3 positions du potentiomètre, avant de téléverser le programme final :

Position	Vout mesurée (V)	T attendue (calcul, °C)

- Téléverser le programme final du §6.2.2.
- Vérifier que le LCD réel affiche bien la température sur la ligne 1 et l'état de luminosité sur la ligne 2.

6.4.4 Tests et comparaison simulation / réel (30 min)

Position	N simulation	N réel	T affichée sur LCD réel (°C)

6.5 Travail élève — Fiche de compte rendu

Questions de synthèse

19. Refaire le calcul complet de a et b pour la température, sans regarder le cours.
20. Pourquoi peut-on afficher directement 2 capteurs sur les 2 lignes du LCD, sans alternance, cette fois-ci ?
21. La température affichée sur le LCD réel est-elle cohérente avec votre calcul manuel (§6.4.3) ? Expliquer un éventuel écart.
22. Le montage LDR de la séance 1 fonctionne-t-il toujours correctement après l'ajout du potentiomètre ? Pourquoi cela ne devrait-il pas le perturber ?

Checklist de fin de séance

- ☐ Calcul de a et b pour la température refait sans le cours
- ☐ Circuit Tinkercad (LDR + température + LCD) monté et simulé
- ☐ Circuit validé par le professeur
- ☐ Câblage réel du potentiomètre réalisé et vérifié au multimètre
- ☐ Programme final téléversé, température ET luminosité affichées sur le LCD réel
- ☐ Tableau comparatif simulation / réel complété
- ☐ Questions de synthèse répondues

6.6 Pour aller plus loin (différenciation)

- Ajouter un seuil : si $T > 30^{\circ}\text{C}$, afficher « ALERTE CHAUD » à la place de l'état de luminosité.
- Tester la fonction `map(Nt, 0, 1023, -10, 40)` pour obtenir directement T, et comparer au calcul avec V.
- Calculer la résolution en $^{\circ}\text{C}$ obtenue par pas d'ADC ($5 / 1023 \text{ V}$, multiplié par la sensibilité a).

6.7 Dépannage — Que faire si...

Symptôme	Cause probable	Solution
La température affichée ne bouge jamais	Curseur non relié à A0	Vérifier le câblage curseur → A0
La luminosité (séance 1) ne fonctionne plus	Fil débranché par erreur en ajoutant le potentiomètre	Revérifier tout le câblage LDR du §5.6
Température incohérente (ex : négative alors que V est grand)	Erreur de signe dans la formule recopiée	Revérifier $T = 10 \times V - 10$
Le LCD affiche des caractères qui se chevauchent	Espaces manquants après une valeur qui a changé de nombre de chiffres	Ajouter des espaces après la valeur affichée

6.8 Grille d'auto-évaluation / compétences

Compétence	Non acquis	En cours	Acquis	Expert
Retrouver une formule de conversion à partir de 2 points connus				
Étendre un montage existant sans le casser				

Compétence	Non acquis	En cours	Acquis	Expert
Afficher 2 grandeurs simultanément sur un LCD 2 lignes				
Comparer simulation et mesure réelle, expliquer un écart				

Prochaine séance : ajout des capteurs d'humidité (A1) et de pression (A2), affichage en alternance sur 2 écrans pour les 4 grandeurs.

7. SÉANCE 3 — Humidité, pression et système complet

Durée : 4 h | Ce qui change par rapport aux séances précédentes

Le montage LDR + température + LCD reste en place. On ajoute aujourd'hui l'humidité (A1) et la pression (A2) : le système utilise maintenant ses 4 entrées analogiques.

Le LCD n'a que 2 lignes pour 4 grandeurs : il faut apprendre à faire alterner automatiquement 2 « écrans » d'affichage.

7.0 Déroulé horaire détaillé (4 h = 240 min)

Horaire	Durée	Activité	Livrable
0h00 – 0h10	10 min	Accueil, rappel séances 1 et 2, objectifs	—
0h10 – 0h30	20 min	Retrouver soi-même les formules H et P (méthode de la droite d'étalonnage)	Formules H et P dérivées
0h30 – 1h00	30 min	PARTIE A1 — Tinkercad : ajout des 2 potentiomètres au circuit existant	Circuit Tinkercad mis à jour
1h00 – 1h30	30 min	PARTIE A2 — Codage : lecture et conversions complètes (4 grandeurs)	Programme de conversion validé
1h30 – 2h00	30 min	PARTIE A3 — Affichage alterné sur 2 écrans LCD	Programme d'affichage validé
2h00 – 2h20	20 min	Trace à la main et interprétation des coefficients	Tableau de trace complété
2h20 – 2h35	15 min	Pause / vérification du travail par le professeur (visa simulation)	Simulation validée
2h35 – 3h00	25 min	PARTIE B1 — Câblage réel des 2 potentiomètres supplémentaires	Montage réel câblé
3h00 – 3h20	20 min	PARTIE B2 — Vérification au multimètre + téléversement	Programme téléversé
3h20 – 3h45	25 min	PARTIE B3 — Tests, comparaison simulation / réel, dépannage	Système complet fonctionnel
3h45 – 4h00	15 min	Fiche de compte rendu et synthèse	Fiche remplie

Toujours la même règle d'or

Aucun câblage réel supplémentaire ni nouveau branchement USB tant que le montage complet (4 capteurs + LCD) n'a pas été simulé avec succès sur Tinkercad et validé par le professeur.

7.1 Retrouver soi-même les formules H et P (20 min)

En utilisant la méthode du §2.2 / §6.1 ($a = (Y_{\max} - Y_{\min}) / (V_{\max} - V_{\min})$, $b = Y_{\min} - a \times V_{\min}$) et les points du cahier des charges (§2), calculer les formules de l'humidité et de la pression :

Grandeur	Point 1 (V=0)	Point 2 (V=5)	a	b	Formule $Y = a \cdot V + b$

Vérification

Vous devez retrouver $H = 16 \cdot V + 10$ et $P = 50 \cdot V + 850$, déjà utilisées dans le cahier des charges (§2). Si ce n'est pas le cas, revérifier le calcul de a et b.

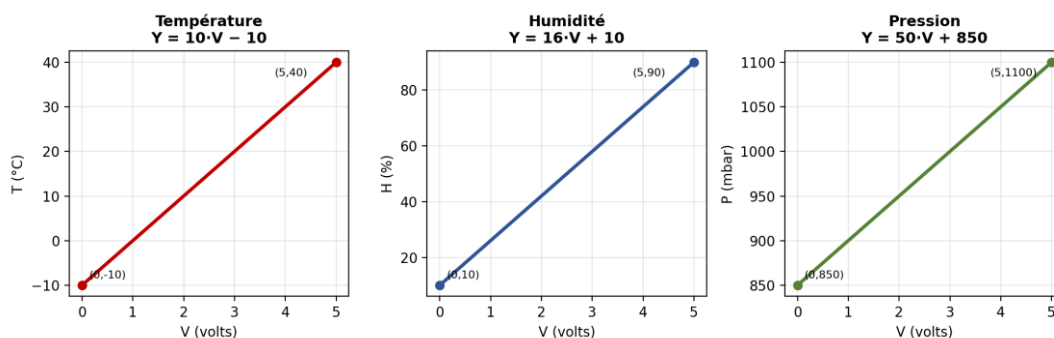


Figure — Les 3 droites d'étalonnage du système : température, humidité, pression.

7.2 PARTIE A — Simulation complète sur Tinkercad (1h30)

7.2.1 Ajout des 2 potentiomètres (30 min)

Reprendre le circuit Tinkercad des séances précédentes (LDR + température + LCD déjà câblés) et ajouter 2 « Rotation Potentiometer » :

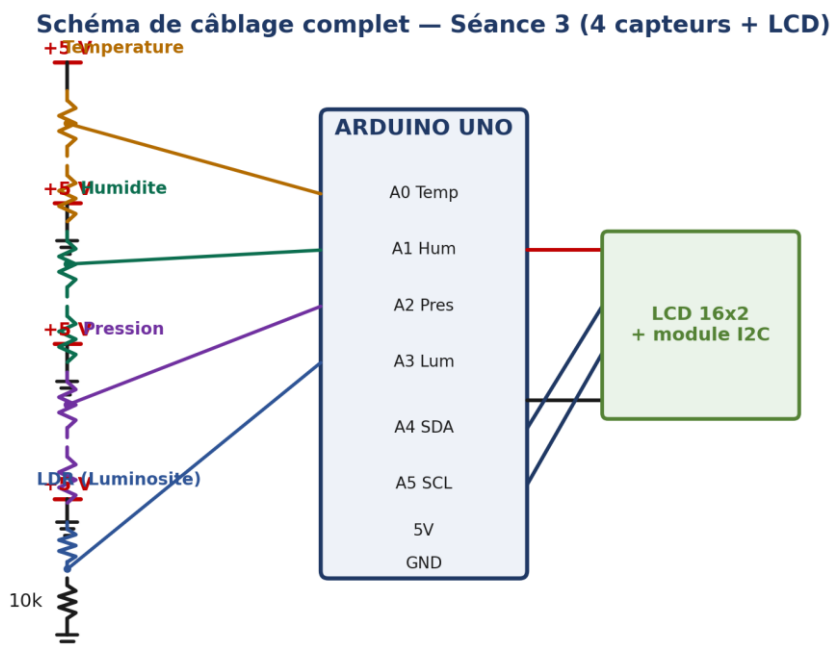


Figure — Schéma de câblage complet de la séance 3 : 4 capteurs + LCD I²C sur l'Arduino Uno.

Capteur	Branchement	Broche Arduino
Potentiomètre température (déjà câblé)	Curseur	A0
Potentiomètre humidité	Curseur	A1
Potentiomètre pression	Curseur	A2
LDR (déjà câblée)	Point milieu	A3

7.2.2 Codage — lecture et conversions complètes (30 min)

```
const int PIN_TEMP = A0, PIN_HUM = A1, PIN_PRES = A2, PIN_LUM = A3;

void setup() {
  Serial.begin(9600);
}

void loop() {
  int Nt = analogRead(PIN_TEMP);
  int Nh = analogRead(PIN_HUM);
  int Np = analogRead(PIN_PRES);
  int Nl = analogRead(PIN_LUM);

  float Vt = Nt * 5.0 / 1023.0;
  float Vh = Nh * 5.0 / 1023.0;
  float Vp = Np * 5.0 / 1023.0;
  float Vl = Nl * 5.0 / 1023.0;

  float T = 10.0 * Vt - 10.0;
  float H = 16.0 * Vh + 10.0;
  float P = 50.0 * Vp + 850.0;
  String L = (Vl < 2.5) ? "SOMBRE" : "CLAIR ";

  Serial.print("T="); Serial.print(T, 1);
  Serial.print(" H="); Serial.print(H, 0);
  Serial.print(" P="); Serial.print(P, 0);
  Serial.print(" L="); Serial.println(L);
  delay(500);
}
```

Tester en simulation : faire varier chaque potentiomètre et vérifier que seule la grandeur correspondante change.

Grandeur	Valeur mini observée	Valeur maxi observée	Cohérente avec le cahier des charges ?

7.2.3 Affichage alterné sur 2 écrans (30 min)

Le LCD n'a que 2 lignes : impossible d'afficher les 4 grandeurs en même temps. On fait alterner deux écrans toutes les 2 secondes : Écran 1 = Température + Humidité, Écran 2 = Pression + Luminosité.

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);
const int PIN_TEMP = A0, PIN_HUM = A1, PIN PRES = A2,
PIN_LUM = A3;
bool afficherEcran1 = true;

void setup() {
  lcd.init();
  lcd.backlight();
  Serial.begin(9600);
}

void loop() {
  int Nt = analogRead(PIN_TEMP);
  int Nh = analogRead(PIN_HUM);
  int Np = analogRead(PIN PRES);
  int Nl = analogRead(PIN_LUM);

  float T = 10.0 * (Nt * 5.0 / 1023.0) - 10.0;
  float H = 16.0 * (Nh * 5.0 / 1023.0) + 10.0;
  float P = 50.0 * (Np * 5.0 / 1023.0) + 850.0;
  String L = ((Nl * 5.0 / 1023.0) < 2.5) ? "SOMBRE" :
"CLAIR ";

  lcd.clear();
  if (afficherEcran1) {
    lcd.setCursor(0, 0);
    lcd.print("Temp: "); lcd.print(T, 1); lcd.print("
C");
    lcd.setCursor(0, 1);
    lcd.print("Hum: "); lcd.print(H, 0); lcd.print("
%");
  } else {
    lcd.setCursor(0, 0);
    lcd.print("Pres: "); lcd.print(P, 0); lcd.print("
mb");
    lcd.setCursor(0, 1);
    lcd.print("Lum: "); lcd.print(L);
  }
  afficherEcran1 = !afficherEcran1;
  delay(2000);
}
```

Instruction	Explication
bool afficherEcran1 = true;	Variable booléenne (vrai/faux) qui mémorise quel écran afficher
lcd.clear();	Efface tout l'écran avant de réafficher, pour éviter les mélanges de caractères

<code>afficherEcran1 = !afficherEcran1;</code>	Inverse la valeur de la variable à chaque tour de boucle (! = « non »)
<code>delay(2000);</code>	Chaque écran reste affiché 2 secondes avant de basculer sur l'autre
Pourquoi <code>lcd.clear()</code> peut faire clignoter l'écran Effacer tout l'écran à chaque boucle peut créer un léger clignotement visible. C'est normal ici. Une solution plus propre (sans <code>clear()</code>) est possible mais plus avancée.	

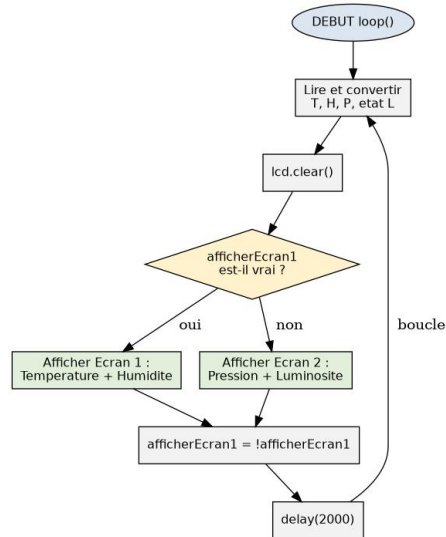


Figure — Algorithme de l'alternance des 2 écrans (test sur `afficherEcran1`).

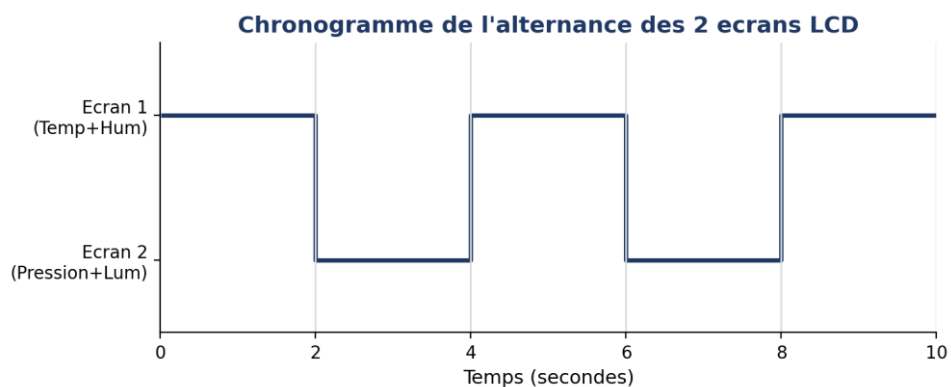


Figure — Chronogramme de l'alternance des 2 écrans dans le temps.

Interprétation — dérouler le programme sans l'ordinateur

Dérouler mentalement 3 passages dans `loop()` en partant de `afficherEcran1 = true`, et compléter :

Passage n°	Valeur de <code>afficherEcran1</code> AVANT affichage	Écran affiché (1 ou 2)	Valeur de <code>afficherEcran1</code> APRÈS

Tester en simulation : les deux écrans doivent alterner toutes les 2 secondes, avec les bonnes valeurs à chaque fois.

7.3 Trace à la main et interprétation des coefficients (20 min)

Formule	Coefficient a (sensibilité)	Coefficient b (valeur à V = 0)
$T = 10 \cdot V - 10$	10 °C par volt	-10 °C
$H = 16 \cdot V + 10$	16 % par volt	10 %
$P = 50 \cdot V + 850$	50 mbar par volt	850 mbar

Calculer à la main V, T, H, P et l'état L pour les valeurs d'ADC suivantes, puis vérifier en simulation :

Valeurs à utiliser

A0
(température)
: N = 614 •
A1 (humidité)
: N = 205 •
A2 (pression)
: N = 921 •
A3
(luminosité) :
N = 300

Entrée	N (donné)	V (calcul)	Grandeur physique (calcul)

- Quel capteur simulé a la plus grande sensibilité ?
- Pourquoi la pression ne peut-elle jamais descendre en dessous de 850 mbar dans ce modèle ?

Validation professeur avant de continuer

Montrer au professeur le circuit Tinkercad complet (4 capteurs + LCD) et les 2 écrans qui alternent correctement.

7.4 PARTIE B — Câblage et tests réels (1h10)

7.4.1 Matériel supplémentaire distribué

Quantité	Matériel
2	Potentiomètres 10 kΩ (humidité, pression)
6	Fils Dupont mâle-mâle supplémentaires

7.4.2 Câblage réel (25 min)

- Potentiomètre humidité : extrémités → +5 V / GND, curseur → A1
- Potentiomètre pression : extrémités → +5 V / GND, curseur → A2

Sécurité

Débrancher l'USB avant toute modification du câblage. Vérifier que le câblage LDR et température des séances précédentes n'a pas été débranché par erreur.

7.4.3 Vérification au multimètre et téléversement (20 min)

Capteur	Tension mesurée (V)

23. Téléverser le programme final du §7.2.3.
24. Vérifier que le LCD réel alterne bien les 2 écrans toutes les 2 secondes.

7.4.4 Tests et comparaison simulation / réel (25 min)

Grandeur	N simulation	N réel	Valeur affichée sur LCD réel

7.5 Travail élève — Fiche de compte rendu

Questions de synthèse

25. Détailler le calcul complet permettant de retrouver $H = 16 \cdot V + 10$ et $P = 50 \cdot V + 850$.
26. Pourquoi utilise-t-on une variable booléenne pour gérer l'alternance des 2 écrans ?
27. Que se passerait-il si on retirait la ligne `lcd.clear()` du programme ?
28. Les 4 grandeurs affichées sur le LCD réel sont-elles cohérentes avec celles obtenues en simulation ? Expliquer un éventuel écart.
29. Le système fonctionne maintenant avec 4 capteurs simultanés : citer une limite matérielle de l'Arduino Uno qui empêcherait d'en ajouter beaucoup plus (nombre de broches analogiques).

Checklist de fin de séance

- ☐ Formules H et P retrouvées soi-même et vérifiées
- ☐ Circuit Tinkercad complet (4 capteurs + LCD) monté et simulé
- ☐ Circuit validé par le professeur
- ☐ Câblage réel des 2 capteurs supplémentaires réalisé et vérifié au multimètre
- ☐ Programme final téléversé sans erreur
- ☐ Les 2 écrans du LCD réel alternent avec des valeurs cohérentes
- ☐ Tableau comparatif simulation / réel complété
- ☐ Questions de synthèse répondues

7.6 Pour aller plus loin (différenciation)

- Réduire le délai d'alternance à 1 seconde et observer l'effet sur la lisibilité.
- Regrouper les 4 lectures dans un tableau (array) et les parcourir avec une boucle for pour les afficher sur le port série.
- Ajouter une alerte : si $H > 80\%$ ET $T > 25^\circ\text{C}$, afficher « RISQUE MOISSURE » à la place de l'écran normal.

7.7 Dépannage — Que faire si...

Symptôme	Cause probable	Solution
Une seule valeur bouge quand on tourne n'importe quel potentiomètre	Deux curseurs branchés sur la même broche	Vérifier A0/A1/A2/A3, un seul fil par broche
L'écran affiche des caractères superposés	<code>lcd.clear()</code> manquant ou mal placé	Vérifier l'appel à <code>lcd.clear()</code> avant chaque affichage
Les 2 écrans ne changent jamais	Variable <code>afficherEcran1</code> non réinitialisée	Vérifier la ligne <code>afficherEcran1 = !afficherEcran1;</code>
H ou P affiche des valeurs négatives ou aberrantes	Erreur dans la formule dérivée au §7.1	Revérifier les coefficients a et b calculés
Le montage des séances précédentes (LDR, température) ne fonctionne plus	Fil débranché par erreur en ajoutant les 2 nouveaux potentiomètres	Revérifier tout le câblage existant avant d'ajouter du nouveau

7.8 Grille d'auto-évaluation / compétences

Compétence	Non acquis	En cours	Acquis	Expert
Retrouver une formule de conversion par la méthode des 2 points				
Câbler et gérer 4 entrées analogiques simultanées				
Utiliser une variable booléenne et l'opérateur ternaire				
Faire alterner un affichage sur un LCD 2 lignes				
Étendre un montage existant sans casser les fonctions précédentes				
Comparer simulation et mesures réelles sur un système complet				

Prochaine séance : intégration finale et ouverture Wi-Fi — envoi des mesures vers un serveur / tableau de bord.

8. SÉANCE 4 — Intégration Wi-Fi et bilan du projet

Durée : 4 h | Objectif de cette séance

Le système mesure déjà 4 grandeurs et les affiche sur le LCD (séances 1 à 3). Aujourd'hui, on ajoute une ouverture réseau : un module Wi-Fi ESP8266 envoie les mesures vers un serveur / tableau de bord, consultable à distance.

Point clé : l'Arduino Uno fonctionne en logique 5 V, mais le module ESP8266 fonctionne en logique 3,3 V. On va réutiliser le diviseur de tension de la séance 1 pour protéger le module !

8.0 Déroulé horaire détaillé (4 h = 240 min)

Horaire	Durée	Activité	Livrable
0h00 – 0h15	15 min	Accueil, présentation de l'extension Wi-Fi, matériel (module ESP8266, résistances)	—
0h15 – 0h45	30 min	Théorie : architecture IoT, notion de requête HTTP, présentation de l'ESP8266	Schéma d'architecture annoté
0h45 – 1h15	30 min	Le diviseur de tension au secours de l'ESP8266 : calcul de l'adaptation 5V → 3,3V	Diviseur de niveau calculé
1h15 – 1h45	30 min	Câblage de l'ESP8266 (alimentation 3,3 V, RX protégée) et test des commandes AT	Communication série validée
1h45 – 2h15	30 min	Connexion au réseau Wi-Fi (commandes AT+CWJAP) et test	Module connecté au réseau
2h15 – 2h30	15 min	Pause / vérification du travail par le professeur	Connexion validée
2h30 – 3h10	40 min	Programmation : envoi des mesures (T, H, P, L) vers un serveur / tableau de bord	Programme d'envoi validé
3h10 – 3h40	30 min	Test du tableau de bord, dépannage	Données visibles à distance
3h40 – 4h00	20 min	Compte rendu final et bilan des 4 séances	Fiche remplie, système complet présenté

Règle d'or de la séance

Ne jamais alimenter le module ESP8266 en 5 V directement : il fonctionne en 3,3 V et peut être détruit. Faire vérifier le câblage par le professeur avant toute mise sous tension.

8.1 Architecture IoT et présentation de l'ESP8266 (30 min)

Jusqu'ici, les mesures restaient affichées localement sur le LCD. Pour les rendre consultables à distance, on ajoute un maillon réseau :

Capteurs -> Arduino Uno (mesure + calcul) -> liaison série -> ESP8266 (Wi-Fi) -> réseau
-> serveur / tableau de bord

L'Arduino Uno ne possède pas de Wi-Fi intégré : il communique avec le module ESP8266 par liaison série (UART), et c'est l'ESP8266 qui gère la connexion au réseau et l'envoi des données, via des commandes texte appelées commandes AT.

Architecture IoT du systeme (Seance 4)



Figure — Architecture IoT complète du système (séance 4).

Élément	Rôle
Arduino Uno	Acquisition des 4 capteurs, calculs, affichage LCD (inchangé)
Module ESP8266	Se connecte au réseau Wi-Fi et envoie les données vers un serveur
Commandes AT	Instructions texte envoyées à l'ESP8266 par liaison série (ex : AT+CWJAP pour se connecter à un réseau)
Serveur / tableau de bord	Reçoit les données et les affiche (page web, service en ligne, ou serveur fourni par le professeur)

Questions d'étude

- Pourquoi ne peut-on pas connecter l'Arduino Uno directement à Internet sans module supplémentaire ?
- Quel est le rôle exact de l'ESP8266 dans cette architecture ?

8.2 Le diviseur de tension au secours de l'ESP8266 (30 min)

L'Arduino Uno communique en logique 5 V (ses broches TX/RX montent à 5 V). L'ESP8266 fonctionne en logique 3,3 V : une tension de 5 V sur son entrée RX peut l'endommager définitivement. On utilise donc un diviseur de tension (§3.4) entre la broche TX de l'Arduino et la broche RX de l'ESP8266.

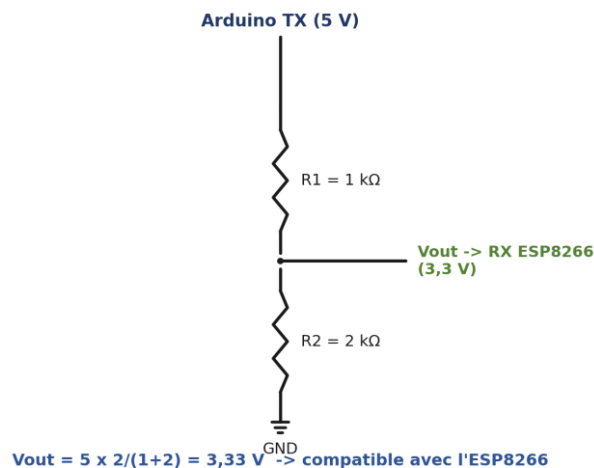


Figure — Diviseur de niveau logique 5V → 3,3V protégeant l'entrée RX de l'ESP8266.

Avec la formule déjà connue (§3.4) : $V_{out} = V_{cc} \times R_2 / (R_1 + R_2)$. Calculer V_{out} pour vérifier qu'il reste compatible avec les 3,3 V de l'ESP8266 :

R1 (Ω)	R2 (Ω)	Vout calculée (V)	Compatible avec 3,3 V ? (oui/non)
Pourquoi seulement dans un sens Ce diviseur ne protège que la ligne Arduino TX → ESP8266 RX (5V → 3,3V). Dans l'autre sens, la broche TX de l'ESP8266 (3,3 V) vers RX de l'Arduino (qui tolère 3,3 V comme un niveau haut) ne nécessite pas de diviseur.			
Alimentation de l'ESP8266 Le module ESP8266 doit être alimenté en 3,3 V (jamais 5 V), en utilisant une sortie 3,3 V dédiée de l'Arduino ou un régulateur externe si la consommation est trop importante pour la broche 3,3V de l'Arduino.			

8.3 Câblage et test des commandes AT (30 min)

Broche ESP8266	Branchement
VCC	3,3 V (jamais 5 V)
GND	GND (commun avec l'Arduino)
RX	TX Arduino, via le diviseur de tension du §8.2
TX	RX Arduino (directement, sans diviseur)

Téléverser un programme minimal qui relaie simplement les commandes tapées dans le moniteur série vers l'ESP8266 (pont série), afin de dialoguer directement avec le module :

```
void setup() {
  Serial.begin(9600); // vers le PC (moniteur serie)
  Serial1.begin(9600); // vers l'ESP8266 (ou SoftwareSerial selon le montage)
}

void loop() {
  if (Serial.available()) Serial1.write(Serial.read());
  if (Serial1.available()) Serial.write(Serial1.read());
}
```

Adapter selon le montage disponible

Selon le matériel du lycée (carte avec un second port série matériel, ou simple ESP8266 relié via SoftwareSerial), la ligne Serial1 peut être remplacée par un objet SoftwareSerial. Se référer à la documentation du module fourni par le professeur.

Dans le moniteur série, taper la commande AT ci-dessous et vérifier la réponse :

Commande envoyée	Réponse obtenue

Commande envoyée	Réponse obtenue

- AT → doit répondre OK (test de communication)
- AT+GMR → affiche la version du firmware du module

8.4 Connexion au réseau Wi-Fi (30 min)

Passer le module en mode « station » (client Wi-Fi) puis se connecter au réseau fourni par le professeur :

<pre>AT+CWMODE=1 AT+CWJAP="nom_du_reseau", "mot_de_passe" AT+CIFSR</pre>	
Commande	Rôle
AT+CWMODE=1	Configure l'ESP8266 en mode station (client)
AT+CWJAP="...", "..."	Connexion au réseau Wi-Fi (nom et mot de passe)
AT+CIFSR	Affiche l'adresse IP obtenue une fois connecté
Adresse IP obtenue	
Validation professeur avant de continuer Montrer au professeur la réponse OK à la connexion et l'adresse IP obtenue avant de passer à l'envoi des données.	

8.5 Envoi des mesures vers un serveur / tableau de bord (40 min)

Le principe : ouvrir une connexion (AT+CIPSTART), envoyer une requête HTTP GET contenant les valeurs mesurées, puis fermer la connexion. Le professeur fournit l'adresse du serveur ou du service de tableau de bord à utiliser.

<pre>AT+CIPSTART="TCP", "adresse_du_serveur", 80 AT+CIPSEND=40 GET /update?t=21.5&h=45&p=1013&l=CLAIR</pre>	
Commande	Rôle
AT+CIPSTART	Ouvre une connexion TCP vers le serveur (adresse et port)
AT+CIPSEND=n	Annonce l'envoi de n caractères, puis attend le texte à envoyer
GET /update?...	Requête HTTP simple transmettant les valeurs dans l'URL

Sur l'Arduino, ces commandes sont envoyées automatiquement depuis le programme final, qui assemble la requête à partir des variables T, H, P et L déjà calculées (séances 1 à 3) :

```
String requete = "GET /update?t=" + String(T, 1)
                + "&h=" + String(H, 0)
                + "&p=" + String(P, 0)
                + "&l=" + L;

Serial1.print("AT+CIPSTART=\"TCP\", \"" + adresseServeur + "\",80\r\n");
delay(500);
Serial1.print("AT+CIPSEND=" + String(requete.length() + 2) + "\r\n");
delay(500);
Serial1.println(requete);
```

String(T, 1)

Convertit un nombre flottant en texte, avec 1 décimale, pour pouvoir l'insérer dans une chaîne de caractères (comme `lcd.print(T,1)`, mais pour construire du texte au lieu d'afficher).

8.6 Test du tableau de bord (30 min)

30. Envoyer une première mesure et vérifier sur le tableau de bord fourni par le professeur que la valeur apparaît.
31. Faire varier un capteur (par exemple la température) et vérifier que la nouvelle valeur remonte bien après quelques secondes.
32. Ajouter un délai raisonnable entre deux envois (par exemple 10 secondes) pour ne pas saturer le réseau.

Grandeur envoyée	Valeur locale (LCD)	Valeur affichée sur le tableau de bord	Cohérentes ?

8.7 Travail élève — Fiche de compte rendu final

Questions de synthèse

33. Pourquoi faut-il un diviseur de tension entre l'Arduino et l'ESP8266, et dans quel sens seulement ?
34. Décrire avec vos mots le chemin complet suivi par une mesure de température, depuis le potentiomètre jusqu'au tableau de bord.
35. Quel est le rôle de la commande AT+CIPSTART ? Et de AT+CIPSEND ?
36. Quelles auraient été les conséquences d'avoir alimenté l'ESP8266 directement en 5 V ?
37. Sur l'ensemble des 4 séances, quelle notion électronique (vue en séance 1) a été réutilisée dans 3 contextes différents ? Lesquels ?

Bilan des 4 séances

Séance	Ce qui a été construit
1	Diviseur de tension (théorie), LDR, bus I ² C, premier affichage LCD
2	Capteur de température, méthode de la droite d'étalonnage
3	Capteurs d'humidité et de pression, système complet à 4 capteurs, affichage alterné
4	Ouverture réseau : ESP8266, protection par diviseur de tension, tableau de bord

Checklist de fin de projet

- ☐ Diviseur de niveau 5V→3,3V calculé et câblé correctement
- ☐ Communication AT avec l'ESP8266 fonctionnelle (réponse OK)
- ☐ Connexion au réseau Wi-Fi réussie, adresse IP obtenue
- ☐ Mesures envoyées et visibles sur le tableau de bord
- ☐ Le système complet (4 capteurs + LCD + Wi-Fi) fonctionne simultanément
- ☐ Questions de synthèse répondues
- ☐ Bilan des 4 séances complété

8.8 Pour aller plus loin (différenciation)

- Envoyer les données vers un vrai service de tableau de bord en ligne (par exemple un compte gratuit sur une plateforme IoT) plutôt qu'un serveur local.
- Ajouter un envoi conditionnel : n'envoyer une nouvelle mesure que si elle a changé significativement, pour économiser la bande passante.
- Remplacer les commandes AT manuelles par une bibliothèque dédiée si l'ESP8266 est utilisé comme carte principale (ESP8266WiFiClient).

8.9 Dépannage — Que faire si...

Symptôme	Cause probable	Solution
Aucune réponse aux commandes AT	Alimentation insuffisante ou mauvais câblage RX/TX	Vérifier 3,3 V stable, vérifier le diviseur de tension
Le module ne se connecte pas au réseau	Mauvais nom de réseau/mot de passe, ou réseau incompatible	Revérifier AT+CWLAP, vérifier que le réseau est en 2,4 GHz
La connexion au serveur échoue (CIPSTART)	Mauvaise adresse ou port, pare-feu	Vérifier l'adresse fournie par le professeur

Symptôme	Cause probable	Solution
Les données n'apparaissent pas sur le tableau de bord	Format de la requête GET incorrect	Comparer la requête envoyée à l'exemple du §8.5
Le module chauffe ou ne répond plus	Alimentation 5 V appliquée par erreur	Couper immédiatement l'alimentation, vérifier le câblage avant de rebrancher

8.10 Grille d'auto-évaluation / compétences

Compétence	Non acquis	En cours	Acquis	Expert
Comprendre une architecture IoT simple				
Adapter un niveau logique avec un diviseur de tension				
Dialoguer avec un module via des commandes AT				
Connecter un module à un réseau Wi-Fi				
Construire et envoyer une requête HTTP simple				
Présenter et argumenter le fonctionnement du système complet				

Fin du projet : le système complet mesure 4 grandeurs, les affiche localement sur un LCD et les transmet à distance via Wi-Fi.